Operatory binarne

Operacje binarne można wykonywać jedynie na zmiennych całkowitych: char, int, long

Operacje binarne zawsze wykonywane są na wszystkich bitach w reprezentacji.

Operatory binarne

Przesunięcie bitowe:
- w lewo <<
- w prawo >>

Przykłady (reprezentacja 8 bitowa):
0011 1011 << 1 → 0111 0110
0000 1011 << 3 → 0101 1000

1011 1011 << 1 → 0111 0110 (zmiana znaku!)
1110 1011 << 1 → 1101 0110

Operatory binarne

Przykłady (reprezentacja 8 bitowa):

0011 1011 >> 1 → 0001 1101

0011 1011 >> 3 → 0000 0111

1011 1011 >> 1 → 1101 1101

1011 1011 >> 3 → 1111 0111

Operatory binarne

Jakiemu działaniu arytmetycznemu odpowiada przesunięcie binarne w lewo?

3 << 1 == 0000 0011 << 1 == 0000 0110 == 6
3 << 2 == 0000 0011 << 2 == 0000 1100 == 12
3 << 3 == 0000 0011 << 3 == 0001 1000 == 24

6 == 3 * 2
12 == 3 * 4
24 == 3 * 8

a << x == a * 2^x

Operatory binarne

Jakiemu działaniu arytmetycznemu odpowiada przesunięcie binarne w lewo?

48 >> 1 == 0011 0000 >> 1 == 0001 1000 == 24
48 >> 2 == 0011 0000 >> 2 == 0000 1100 == 12
48 >> 3 == 0011 0000 >> 3 == 0000 0110 == 6

24 == 48 / 2
12 == 48 / 4
6 == 48 / 8

$a >> x == a / 2^x$

Operatory binarne

Iloczyn binarny - &

Przykład (reprezentacja 8 bitowa)

```
7 & 5 == 0000 0111
             &
         0000 0011
        ----------
         0000 0011
```

Iloczyn binarny wykonywany jest na bitach z odpowiadających sobie pozycji.

Operatory binarne

Suma binarna - I

Przykład (reprezentacja 8 bitowa)

```
21 | 5 == 0001 0101
               I
          0000 0011
         -----------
          0001 0111
```

Suma binarna wykonywana jest na bitach z odpowiadających sobie pozycji.

Operatory binarne

Różnica symetryczna, XOR - ^

Przykład (reprezentacja 8 bitowa)

```
21 ^ 5 == 0001 0101
              ^
          0000 0011
         -----------
          0001 0110
```

Różnica symetryczna wykonywana jest na bitach z odpowiadających sobie pozycji.

Operatory binarne

Negacja - ~

Przykład (reprezentacja 8 bitowa)

~21 == 0001 0101

~

1110 1010

Negacja wykonywana jest na wszystkich bitach.

Operatory binarne

Możliwe jest stosowanie operatorów złożonych:

- <<=
- >>=
- &=
- |=
- ^=

Zliczanie jedynek |

```
int wartosc;
cout << ”Podaj wartość całkowitą.” << endl;
cin >> wartosc;

int suma_bitow = 0;
int maska = 1;
for (int i = 0; i < 32; ++i) {
        if (wartość & maska) suma_bitow++;
        maska <<= 1;
}
cout << ”Liczba jedynek wynosci: ” << suma_bitow;
```

Zliczanie jedynek ||

Niech dana będzie liczba 8 bitowa o wartości dziesiętnej 211.
Jej reprezentacja bitowa to: 1101 0011

Co się stanie jeśli wyznaczymy iloczyn binarny ten wartości z wartością mniejszą o 1?

x ? 211 & 210

x == 1101 0011 & 1101 0010 == 1101 0010
Znika jedna jedynka binarna. Dalej …
x == 1101 0010 & 1101 0001 == 1101 0000
Znika kolejna jedynka binarna. Dalej …
x == 1101 0000 & 1100 1111 == 1100 0000
Znika kolejna jedynka binarna. ltd. aż x == 0

Zliczanie jedynek ||

```
int wartosc;
cout << ”Podaj wartość całkowitą.” << endl;
cin >> wartosc;

int liczba_jedynek = 0;
while (wartość) {
        wartość = wartosc & (wartosc – 1);
        ++liczba_jedynek;
}
cout << ”Liczba jedynek wynosi: ” << liczba_jedynek;
```

Zliczanie jedynek |||

Niech dana będzie liczba 8 bitowa o wartości dziesiętnej 211.
Jej reprezentacja bitowa to: 1101 0011

W jakich najmniejszych blokach można policzyć jedynki?
W blokach 2 bitowych:

11 01 00 11
1+1 0+1 0+0 1+1
2 1 0 2
---
10 01 00 10

W kolejnym kroku należy zliczyć jedynki w blokach 4 bitowych.
10 01 00 10

Zliczanie jedynek |||

W kolejnych krokach należy sumować sumy cząstkowe.

Dla bloku 4 bitowego:

```
10 01 00 10
2  + 1  0 + 2
-------------------
00 11 00 10
```

Dla bloku 8 bitowego

```
00 11 00 10
3  +  2
-------------------
0000 0101
```

Wartość dziesiętna 5, czyli jest to szukany wynik 3 krokach!

Zliczanie jedynek |||

```
int wartosc;
cin >> wartosc;
const int maska_1a = 0x55555555;
const int maska_1b = 0xAAAAAAAA;

int temp_1 = wartosc & maska_1a;
int temp_2 = wartosc & maska_1b;
wartosc = temp_1 + (temp2 >> 1);

const int maska_2a = 0x33333333;
const int maska_2b = 0xCCCCCC;

temp_1 = wartosc & maska_2a;
temp_2 = wartosc & maska_2b;
wartosc = temp_1 + (temp2 >> 2);
```

Zliczanie jedynek |||

```
const int maska_3a = 0x0F0F0F0F;
const int maska_3b = 0xF0F0F0F0;

temp_1 = wartosc & maska_1a;
temp_2 = wartosc & maska_1b;
wartosc = temp_1 + (temp2 >> 4);

const int maska_4a = 0x00FF00FF;
const int maska_4b = 0xFF00FF00;

temp_1 = wartosc & maska_1a;
temp_2 = wartosc & maska_1b;
wartosc = temp_1 + (temp2 >> 8);
```

Zliczanie jedynek |||

```
const int maska_5a = 0x0000FFFF;
const int maska_5b = 0xFFFF0000;

temp_1 = wartosc & maska_1a;
temp_2 = wartosc & maska_1b;
wartosc = temp_1 + (temp2 >> 16);

cout << ”Liczba jedynek wynosi: ” << wartosc <<endl;
```

Bitowa reprezentacja zbiorów

Jeden bit wskazuje obecność elementu w zbiorze.

Przykład:
Zbiór liter zawiera następujące litery: a, d, f, m, o, z
Bitowa reprezentacja tego zbioru:

a	b	c	d	e	f	g	h	i	j	k	l	m	n	o	p	r	s	t	u	w	x	y	z
1	0	0	1	0	1	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1

- iloczyn zbiorów: $P = A \cap B \rightarrow P = A \,\&\, B$
- suma zbiorów: $P = A \cup B \rightarrow P = A \mid B$
- dopełnienie zbioru: $P = A' \rightarrow P = \sim A$

Sito Erastotenesa

Algorytm wyszukujący wszystkie liczy pierwsze z zadanego zakresu.

Kroki:

- Pomiń 0 i 1,
- Wykreśl wszystkie wielokrotności liczby 3 ze zbioru,
- Przejdź do pierwszej niewykreślonej liczby (liczba pierwsza) i wykreśl wszystkie jej wielokrotności ze zbioru,
- Powtarzaj poprzedni punkt dopóki kwadrat liczby pierwszej jest mniejszy lub równy największej wartości w zbiorze.

Sito Erastotenesa

```
int range;
cin >> range;
unsigned int sieve = 0xFFFFFFFC; // Najmłodsze bity: 1100 – 0 oznacza że indeks nie jest liczbą pierwsza
int prime = 2;

while (prime * prime <= range) {
  for (int multiple = 2 * prime; multple <= range; multiple += prime)
     sieve &= ~(1 << multiple);

  do {                                  // Znajdź kolejną liczbę pierwszą.
     prime++;
  } while ( !(sieve & (1 << prime)) );
}

for (int i = 0; i < 32; ++i)
  if (sieve & (1 << i)) cout << i << " ";
```